COURS DE LANGAGE C — Le Guide de ARIS NK

> *"Yo les devs ! Moi c'est ARIS NK, et aujourd'hui on va apprendre le C ensemble. Pas de panique, je vais tout expliquer comme si on était autour d'un café. Même si t'as jamais codé de ta vie, tu vas comprendre. Promis."*

Table des matières

- 0 — Introduction
  - 0.1 Histoire du langage C
  - 0.2 Pourquoi apprendre le C ?
  - 0.3 Utilisation du langage C
- 1 — Installation du compilateur GCC et de l'éditeur de code
  - 1.1 Installation du compilateur GCC
  - 1.2 Installation de l'éditeur de code (VS Code)
  - 1.3 Configuration de l'environnement de développement
- 2 — Écriture et exécution du 1er programme
  - 2.1 La fonction main
  - 2.2 Printf
  - 2.3 Compilation et exécution
- 3 — Astuces
  - 3.1 Astuce pour des retours à la ligne

---

0 — Introduction

0.1 Histoire du langage C

Imagine : on est en **1972**, à une époque où les ordinateurs étaient gros comme des armoires et où personne n'avait de smartphone. Deux mecs brillants, **Dennis Ritchie** et **Ken Thompson**, bossaient aux **Bell Labs** (les labos de recherche d'AT&T aux États-Unis).

Leur problème ? Ils voulaient réécrire le système d'exploitation **UNIX**, mais les langages de l'époque étaient soit :

- **Trop bas niveau** (l'assembleur) → super pénible à écrire, différent pour chaque machine.
- **Trop haut niveau** → trop lent pour un système d'exploitation.

Ils avaient besoin d'un truc **au milieu** : un langage qui soit :

- Assez proche de la machine pour être rapide
- Assez lisible pour qu'un humain puisse coder sans s'arracher les cheveux

Et boom , le **langage C** est né !

> **Fun fact** : Il s'appelle "C" parce qu'il succède au langage "B" (créé par Ken Thompson). Oui, c'est littéralement aussi simple que ça. Le langage B venait lui-même d'un langage appelé BCPL. Pas de mystère, juste l'alphabet

La timeline rapide :

Année	Événement
1972	Dennis Ritchie crée le langage C aux Bell Labs
1973	UNIX est réécrit en C (avant c'était en assembleur)
1978	Le livre **"The C Programming Language"** sort (la bible du C, par Kernighan & Ritchie)
1989	Le C est normalisé par l'ANSI → on parle de **C89** ou **ANSI C**
1999	Nouvelle norme : **C99** (avec des améliorations sympas)
2011	**C11** sort (la version moderne)
2023	**C23** est la dernière norme en date

Le C a **plus de 50 ans** et il est toujours massivement utilisé. Quand un langage survit aussi longtemps, c'est qu'il a quelque chose de spécial.

0.2 Pourquoi apprendre le C ?

Tu te dis peut-être : *"Pourquoi apprendre un langage de 1972 alors que y'a Python, JavaScript, et tout le reste ?"*

Excellente question ! Voici pourquoi le C reste **indispensable** :

1. Tu comprends VRAIMENT comment un ordinateur fonctionne
Le C te force à comprendre ce qui se passe **sous le capot** :

- Comment la **mémoire** fonctionne (RAM, pile, tas…)
- Comment l'ordinateur **stocke** des données
- Comment le processeur **exécute** des instructions

Quand tu codes en Python, le langage fait plein de trucs pour toi en arrière-plan. En C, **c'est toi le chef**. Rien ne se passe sans que tu le décides.

> **Analogie** : Apprendre Python sans connaître le C, c'est comme conduire une voiture automatique sans jamais avoir compris comment un moteur fonctionne. Tu sais conduire, mais tu ne comprends pas pourquoi ça roule.

2. Le C est ULTRA rapide Le C produit du code qui tourne **presque aussi vite** que de l'assembleur. Pourquoi ? Parce qu'il est **compilé** directement en langage machine (les 0 et les 1 que le processeur comprend).

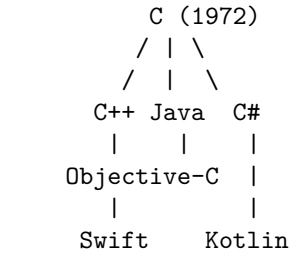
Comparaison de vitesse (pour donner une idée) :

```
Assembleur   → Le plus rapide, mais galère à écrire
C            → Presque aussi rapide, bien plus facile à lire  ← ON EST LÀ
C++          → Rapide aussi, mais plus complexe
Java         → Correct, mais plus lent
Python       → Facile à écrire, mais beaucoup plus lent
```

3. Le C est PARTOUT Tout le monde utilise du C sans le savoir :

- **Ton système d'exploitation** (Windows, Linux, macOS) → écrit en C
- **Ton navigateur web** → des parties critiques en C/C++
- **Les jeux vidéo** → les moteurs graphiques utilisent du C/C++
- **Ton téléphone** → le noyau Android est en C
- **Ton frigo connecté / ta voiture** → programmés en C (systèmes embarqués)

4. C'est la base de presque tous les langages modernes Regarde cette "famille" :



Si tu comprends le C, apprendre **C++, Java, C#, et beaucoup d'autres** sera BEAUCOUP plus facile. La syntaxe est similaire, les concepts sont les mêmes.

5. Le C sur le marché du travail Le C est demandé dans des secteurs qui paient **très bien** :

- Systèmes embarqués (voitures, avions, appareils médicaux)
- Jeux vidéo (moteurs de jeux)
- Cybersécurité (reverse engineering, exploits)
- Systèmes d'exploitation
- Télécommunications

0.3 Utilisation du langage C

Concrètement, le C est utilisé dans **6 grands domaines** :

1. Les systèmes d'exploitation C'est LE domaine du C. Les OS sont écrits en C parce qu'ils ont besoin d'**accéder directement au matériel** (processeur, mémoire, disque dur…).

- **Linux** → le noyau est entièrement en C (~27 millions de lignes !)
- **Windows** → le noyau est en C et C++
- **macOS / iOS** → basés sur Darwin, écrit en C

2. Les systèmes embarqués Un système embarqué, c'est un mini-ordinateur intégré dans un objet du quotidien :

- Le micro-contrôleur de ta machine à laver
- Le système de freinage ABS de ta voiture
- Les capteurs dans les drones
- Les satellites en orbite

Ces appareils ont **très peu de mémoire** et de puissance. Le C est parfait parce qu'il consomme très peu de ressources.

3. Les jeux vidéo et le graphisme Les moteurs de jeux comme **Unreal Engine** utilisent du C/C++ pour les parties critiques où la performance est essentielle (rendu graphique, physique, gestion mémoire).

4. Les bases de données Les SGBD (Systèmes de Gestion de Bases de Données) les plus utilisés au monde sont écrits en C :

- **MySQL**
- **PostgreSQL**
- **SQLite**

La raison ? Quand tu gères des millions de requêtes par seconde, chaque milliseconde compte.

5. Les langages de programmation eux-mêmes Tu utilises Python ? Eh bien, l'interpréteur Python (CPython) est écrit… **en C** !

Pareil pour :

- **PHP** → écrit en C
- **Ruby** → écrit en C
- **Node.js** (le moteur V8) → écrit en C++

Le C est littéralement le langage qui fait tourner les autres langages.

6. La cybersécurité Comprendre le C est **obligatoire** en cybersécurité pour :

- Analyser des malwares (reverse engineering)
- Trouver des failles (buffer overflow, etc.)
- Développer des outils de sécurité

---

1 — Installation du compilateur GCC et de l'éditeur de code

Avant de coder, on a besoin de **deux outils** :

1. **Un compilateur** → il traduit ton code C en langage machine (les 0 et 1 que comprend ton proc)
2. **Un éditeur de code** → c'est là où tu vas écrire ton code (comme un Word, mais pour le code)

> **Analogie** : L'éditeur de code, c'est ton **établi** où tu construis ton meuble. Le compilateur, c'est la **machine** qui transforme tes planches en un meuble fini. Sans la machine, tu as juste des planches. Sans l'établi, tu ne peux pas travailler confortablement.

1.1 Installation du compilateur GCC

GCC (GNU Compiler Collection) est le compilateur le plus populaire et il est **gratuit**. On va l'installer selon ton système d'exploitation.

Sur Windows Sur Windows, GCC n'existe pas directement. On utilise **MinGW** (Minimalist GNU for Windows) qui embarque GCC.

Étape 1 : Télécharger MSYS2

1. Va sur le site officiel : https://www.msys2.org/
2. Clique sur le bouton de téléchargement (le `.exe`)
3. Lance l'installateur et suis les étapes (laisse tout par défaut, clique "Next" partout)

Étape 2 : Installer GCC via MSYS2

1. Ouvre le terminal **MSYS2 UCRT64** (cherche-le dans le menu Démarrer)
2. Tape cette commande et appuie sur Entrée :

```
pacman -S mingw-w64-ucrt-x86_64-gcc
```

1. Quand on te demande `[Y/n]`, tape `Y` et appuie sur Entrée
2. Attends que ça s'installe

Étape 3 : Ajouter GCC au PATH

Le **PATH**, c'est une liste d'endroits où Windows cherche les programmes. Il faut dire à Windows où trouver GCC.

1. Appuie sur `Win + R`, tape `sysdm.cpl` et appuie sur Entrée
2. Va dans l'onglet **"Paramètres système avancés"**
3. Clique sur **"Variables d'environnement"**
4. Dans la section **"Variables système"**, trouve la variable `Path` et double-clique dessus
5. Clique sur **"Nouveau"** et ajoute ce chemin :

```
C:\msys64\ucrt64\bin
```

1. Clique OK, OK, OK (ferme toutes les fenêtres)

Étape 4 : Vérifier l'installation

Ouvre un **nouveau** terminal (CMD ou PowerShell) et tape :

```
gcc --version
```

Si tu vois quelque chose comme ça, **bravo, c'est installé !**

```
gcc (Rev3, Built by MSYS2 project) 13.2.0
Copyright (C) 2023 Free Software Foundation, Inc.
```

> **Attention** : Si ça dit `gcc n'est pas reconnu comme une commande interne`, c'est que le PATH n'est pas bien configuré. Revérifie l'étape 3, et surtout **ouvre un NOUVEAU terminal** (les anciens ne prennent pas en compte les changements).

Sur Linux (Ubuntu / Debian) Sur Linux, c'est **beaucoup plus simple** (merci Linux).

Ouvre un terminal et tape :

```
sudo apt update
sudo apt install gcc
```

C'est tout. Sérieusement.

Pour vérifier :

```
gcc --version
```

> **Note** : Si tu es sur **Fedora**, utilise `sudo dnf install gcc`. Sur **Arch Linux**, utilise `sudo pacman -S gcc`.

Sur macOS Sur macOS, GCC est inclus dans les **Xcode Command Line Tools**.

Ouvre un terminal et tape :

```
xcode-select --install
```

Une fenêtre va s'ouvrir, clique sur **"Installer"**. Attends que ça se termine.

Pour vérifier :

```
gcc --version
```

> **Note** : Techniquement, macOS utilise **Clang** (un autre compilateur) qui se fait passer pour GCC. Pas de panique, ça fonctionne exactement pareil pour ce qu'on fait.

1.2 Installation de l'éditeur de code (VS Code)

On va utiliser **Visual Studio Code** (VS Code), l'éditeur de code **le plus populaire au monde**. C'est gratuit, open source, et super puissant grâce à ses extensions.

> Ne confonds pas **Visual Studio Code** (léger, gratuit) avec **Visual Studio** (un gros IDE lourd de Microsoft). Ce sont deux logiciels différents !

Étape 1 : Télécharger VS Code

1. Va sur : https://code.visualstudio.com/
2. Clique sur le gros bouton bleu **"Download"**
3. Lance l'installateur
4. **Coche bien la case** : `Ajouter au PATH` pendant l'installation (c'est important !)

Étape 2 : Premier lancement

1. Ouvre VS Code
2. Tu vas voir une page de bienvenue — prends 2 min pour regarder, puis ferme-la
3. Tu es prêt(e) !

1.3 Configuration de l'environnement de développement

Maintenant qu'on a le compilateur ET l'éditeur, on va les faire **travailler ensemble**.

Installer l'extension C/C++ dans VS Code

1. Ouvre VS Code
2. Clique sur l'icône **Extensions** dans la barre latérale gauche (ou tape `Ctrl + Shift + X`)
3. Dans la barre de recherche, tape : `C/C++`
4. Installe l'extension **"C/C++"** de **Microsoft** (celle avec le plus de téléchargements)

Cette extension te donne :

- La **coloration syntaxique** (le code est coloré, bien plus lisible)
- L'**autocomplétion** (VS Code te suggère du code pendant que tu tapes)
- La **détection d'erreurs** en temps réel (les erreurs sont soulignées en rouge)
- Le **débogage** (pour trouver tes bugs étape par étape)

Créer ton dossier de travail C'est une bonne pratique de créer un dossier dédié à tes projets C :

1. Crée un dossier quelque part sur ton PC, par exemple : `C:\Projets\Bootcamp_C\`
2. Dans VS Code, va dans **Fichier → Ouvrir le dossier** et sélectionne ton dossier
3. VS Code va te demander si tu fais confiance aux auteurs → clique **Oui**

> **Conseil** : Évite les chemins avec des espaces ou des caractères spéciaux (accents, etc.). Préfère `Bootcamp_C` plutôt que `Bootcamp C` ou `Bôôtcamp Ç`. Ça t'évitera des maux de tête plus tard.

Vérifier que tout fonctionne ensemble Pour vérifier que GCC et VS Code sont bien connectés :

1. Dans VS Code, ouvre le **terminal intégré** : `Ctrl + ù` (ou **Terminal → Nouveau terminal**)
2. Tape :

```
gcc --version
```

Si tu vois la version de GCC s'afficher, **tout est bon !**

Si ça ne marche pas, vérifie que :

- GCC est bien installé (retourne à la section 1.1)
- Le PATH est bien configuré
- Tu as **redémarré** VS Code après avoir configuré le PATH

---

2 — Écriture et exécution du 1er programme

C'est le moment le plus excitant : **on va écrire notre tout premier programme en C !**

La tradition veut que le premier programme de tout développeur affiche `Hello, World!` à l'écran. On va respecter cette tradition sacrée.

2.1 La fonction `main`

En C, **tout programme commence par la fonction `main`**. C'est le **point d'entrée** de ton programme. Quand tu lances ton programme, l'ordinateur cherche la fonction `main` et commence à exécuter le code qui est à l'intérieur.

> **Analogie** : Imagine que ton programme est un livre. La fonction `main`, c'est la **première page** que le lecteur (l'ordinateur) ouvre. Sans première page, pas de livre. Sans `main`, pas de programme.

Voici la structure minimale d'un programme en C :

```c
#include <stdio.h>

int main() {
    // Ton code va ici
    return 0;
}
```

Décomposons **chaque ligne** :

```c
#include <stdio.h>
```

```c
#include <stdio.h>
```

- `#include` dit au compilateur : *"Hé, va me chercher ce fichier !"*
- `<stdio.h>` c'est le fichier **Standard Input/Output Header** (En-tête d'Entrée/Sortie Standard)
- Ce fichier contient les fonctions pour **afficher du texte** à l'écran (`printf`) et pour **lire** ce que l'utilisateur tape au clavier (`scanf`)

> **Analogie** : C'est comme quand tu importes une bibliothèque en Python avec `import`. En C, la syntaxe est `#include`. Le fichier `stdio.h` est la bibliothèque de base pour parler à l'utilisateur.

Sans cette ligne, tu ne pourras pas utiliser `printf`. Le compilateur te dira : *"C'est quoi printf ? Connais pas mec."*

```
int main() { }
```

```
int main() {
    // le code ici
}
```

Décortiquons :

Élément	Signification
`int`	Le **type de retour**. La fonction `main` renvoie un **entier** (integer). C'est le code de sortie du programme.
`main`	Le **nom** de la fonction. C'est toujours `main`, pas `Main`, pas `MAIN`, pas `start`. Juste `main`.
`()`	Les **parenthèses** contiennent les paramètres. Pour l'instant, on les laisse vides.
`{ }`	Les **accolades** délimitent le **corps** de la fonction. Tout le code entre `{` et `}` fait partie de la fonction.

```
return 0;
```

```
return 0;
```

- `return` dit : *"Je renvoie une valeur et je termine la fonction."*
- `0` signifie : *"Tout s'est bien passé, pas d'erreur."*

C'est une **convention universelle** en informatique :

- `0` = Succès, tout va bien
- Autre chose que `0` (1, 2, -1…) = Erreur

> Quand tu lances un programme depuis un terminal et qu'il plante, le code de retour sera différent de 0. C'est comme ça que le système d'exploitation sait si le programme a réussi ou échoué.

2.2 `printf`

`printf` est la fonction qui te permet d'**afficher du texte à l'écran**. C'est la première fonction que tout développeur C apprend, et tu vas l'utiliser **tout le temps**.

Le nom vient de : **print** (afficher) + **f** (formatted, formaté).

Voici notre programme complet — le fameux **Hello World** :

```c
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

Analysons la ligne `printf` :

```c
printf("Hello, World!");
```

Élément	Signification
`printf`	Le nom de la fonction
`( )`	Les parenthèses entourent l'**argument** (ce qu'on donne à la fonction)
`"Hello, World!"`	Le **texte à afficher** — il DOIT être entre guillemets doubles `"`
`;`	Le **point-virgule** termine l'instruction. **C'est OBLIGATOIRE en C.**

> **LE POINT-VIRGULE** — L'erreur n°1 des débutants en C, c'est d'**oublier le point-virgule ;** à la fin d'une ligne. Si ton programme ne compile pas et que tu ne comprends pas pourquoi, vérifie tes points-virgules. 90% du temps, c'est ça le problème.

2.3 Compilation et exécution

Contrairement à Python qui exécute ton code **ligne par ligne** (interprété), le C doit d'abord être **transformé** en langage machine. C'est ce qu'on appelle la **compilation**.

Le processus en image

```
Code source (.c)     →     Compilation (gcc)     →     Exécutable (.exe)
  (ce que toi tu lis)        (traduction en binaire)       (ce que la machine lit)
```

Étape 1 : Créer le fichier

1. Dans VS Code, crée un nouveau fichier : `Ctrl + N`
2. Sauvegarde-le sous le nom `hello.c` : `Ctrl + S`
   - **Attention** : L'extension doit être `.c` (pas `.txt`, pas `.py`, c'est `.c`)
3. Tape le code suivant :

```c
#include <stdio.h>

int main() {
```

```
    printf("Hello, World!");
    return 0;
}
```

1. Sauvegarde (`Ctrl + S`)

Étape 2 : Compiler Ouvre le terminal intégré de VS Code (`Ctrl + ù`) et tape :

```
gcc hello.c -o hello
```

Décortiquons cette commande :

Élément	Signification
`gcc`	On appelle le compilateur GCC
`hello.c`	Le fichier source à compiler
`-o`	L'option "output" (sortie) — on spécifie le nom du fichier de sortie
`hello`	Le nom de l'exécutable qu'on veut créer

Si **aucun message n'apparaît** → C'est bon signe ! GCC ne dit rien quand tout va bien. Pas de nouvelle, bonne nouvelle.

Si des **messages d'erreur** apparaissent → Lis-les attentivement. Le compilateur te dit exactement quelle ligne pose problème.

Exemple d'erreur classique :

```
hello.c:4:30: error: expected ';' before 'return'
```

Ça veut dire : *"À la ligne 4, colonne 30, il manque un point-virgule avant le mot `return`."* → Le compilateur est ton **ami**, il te dit exactement ce qui ne va pas !

Étape 3 : Exécuter Maintenant, on lance notre programme :

Sur Windows :

```
.\hello.exe
```

Sur Linux / macOS :

```
./hello
```

Et tu devrais voir s'afficher :

```
Hello, World!
```

FÉLICITATIONS ! Tu viens d'écrire, compiler et exécuter ton tout premier programme en C !

Récap rapide : La procédure sera toujours la même pour chaque programme :

1. **Écrire** le code dans un fichier `.c`
2. **Compiler** avec `gcc monfichier.c -o monprogramme`
3. **Exécuter** avec `./monprogramme` (ou `.\monprogramme.exe` sur Windows)

---

3 — Astuces

3.1 Astuce pour des retours à la ligne

Tu as peut-être remarqué un truc bizarre : quand on a affiché `Hello, World!`, le texte et le curseur du terminal étaient **collés** sur la même ligne. Pas très joli, non ?

En C, pour faire un **retour à la ligne**, on utilise le caractère spécial `\n`. Le `\` est appelé un **caractère d'échappement**, et `n` veut dire **new line** (nouvelle ligne).

Exemple SANS `\n`

```c
#include <stdio.h>

int main() {
    printf("Bonjour");
    printf("les devs");
    return 0;
}
```

Résultat :

```
Bonjourles devs
```

Tout est collé ! Pas top.

Exemple AVEC `\n`

```c
#include <stdio.h>

int main() {
    printf("Bonjour\n");
    printf("les devs\n");
    return 0;
}
```

Résultat :

```
Bonjour
les devs
```

Beaucoup mieux !

Tu peux même mettre plusieurs \n en plein milieu d'un texte

```c
#include <stdio.h>

int main() {
    printf("Ligne 1\nLigne 2\nLigne 3\n");
    return 0;
}
```

Résultat :

```
Ligne 1
Ligne 2
Ligne 3
```

Les autres caractères d'échappement utiles Tant qu'on y est, voici d'autres caractères spéciaux que tu pourras utiliser :

Caractère	Effet	Exemple
`\n`	Retour à la ligne	`printf("Hello\nWorld");`
`\t`	Tabulation (indentation)	`printf("Col1\tCol2");`
`\\`	Afficher un antislash `\`	`printf("Chemin: C:\\Users");`
`\"`	Afficher un guillemet double `"`	`printf("Il a dit \"salut\"");`
`\0`	Caractère nul (fin de chaîne)	(on verra ça plus tard)

> **Pourquoi \\ et \" ?** Parce que `\` et `"` ont déjà une signification spéciale en C. Le `\` introduit un caractère d'échappement, et `"` délimite les chaînes de caractères. Donc pour les afficher littéralement, on les "échappe" en les doublant.

Exercice pour toi Essaie d'écrire un programme qui affiche :

```
*********
*       *
* HELLO *
*       *
*********
```

Voir la solution

```c
#include <stdio.h>

int main() {
    printf("*********\n");
    printf("*       *\n");
    printf("* HELLO *\n");
    printf("*       *\n");
    printf("*********\n");
    return 0;
}
```

Conclusion

Bravo, tu viens de terminer le **chapitre d'introduction** !

Récapitulons ce qu'on a appris :

- L'**histoire** du C et pourquoi il a été créé
- **Pourquoi** le C est toujours pertinent et puissant
- **Où** le C est utilisé dans le monde réel
- Comment **installer** GCC et VS Code
- Comment **écrire**, **compiler** et **exécuter** son premier programme
- Comment utiliser `printf` et le retour à la ligne `\n`

Tu as maintenant les **fondations**. Dans les prochains chapitres, on va rentrer dans le vif du sujet avec les **variables**, les **types de données**, les **conditions**, les **boucles**… Et là, ça va devenir **vraiment fun** !

> *"Le voyage de mille lignes de code commence par un simple `Hello, World!`"* — Un dev sage (probablement)

Cours rédigé avec par ARIS NK. Si t'as des questions, hésite pas !